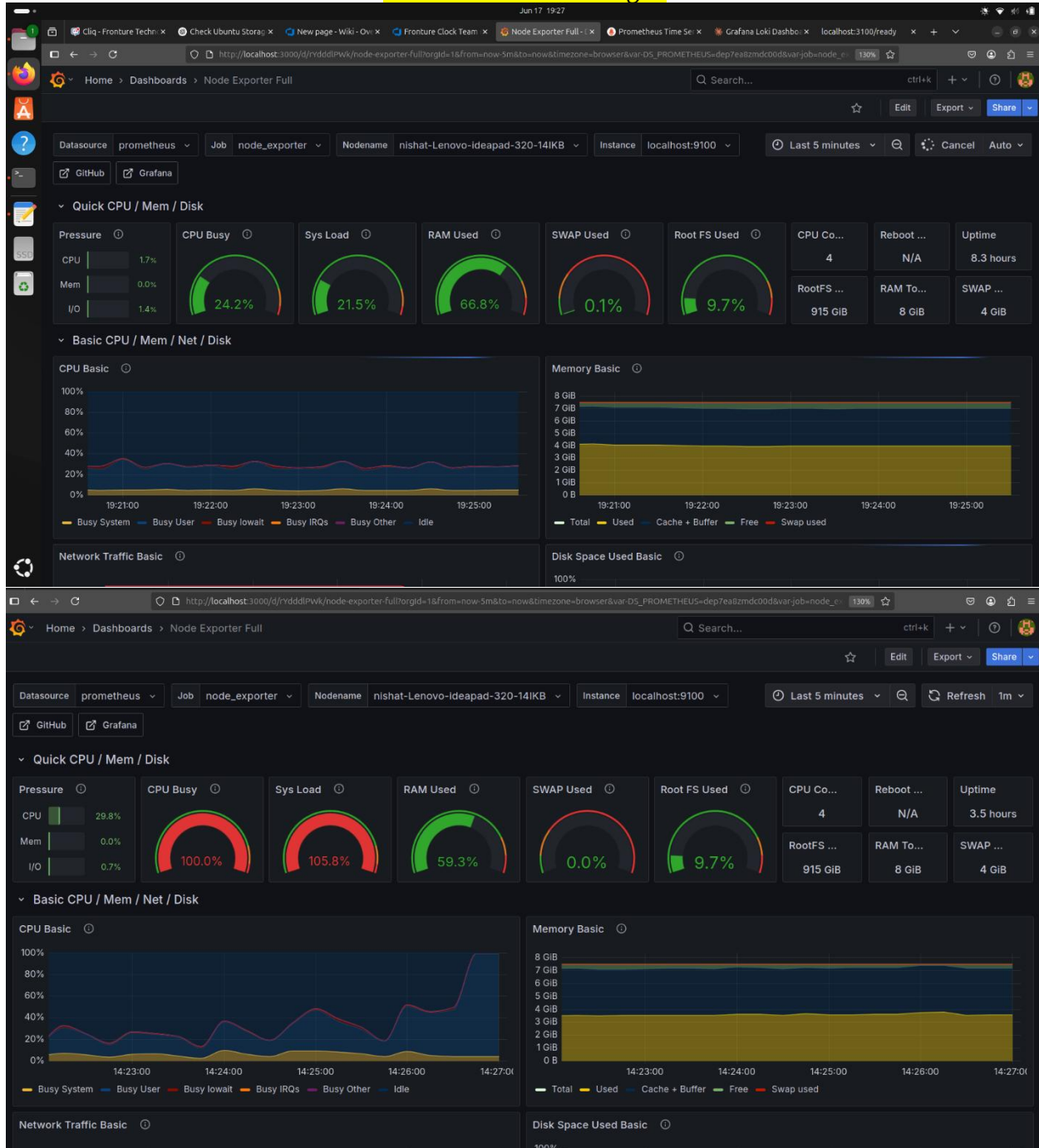


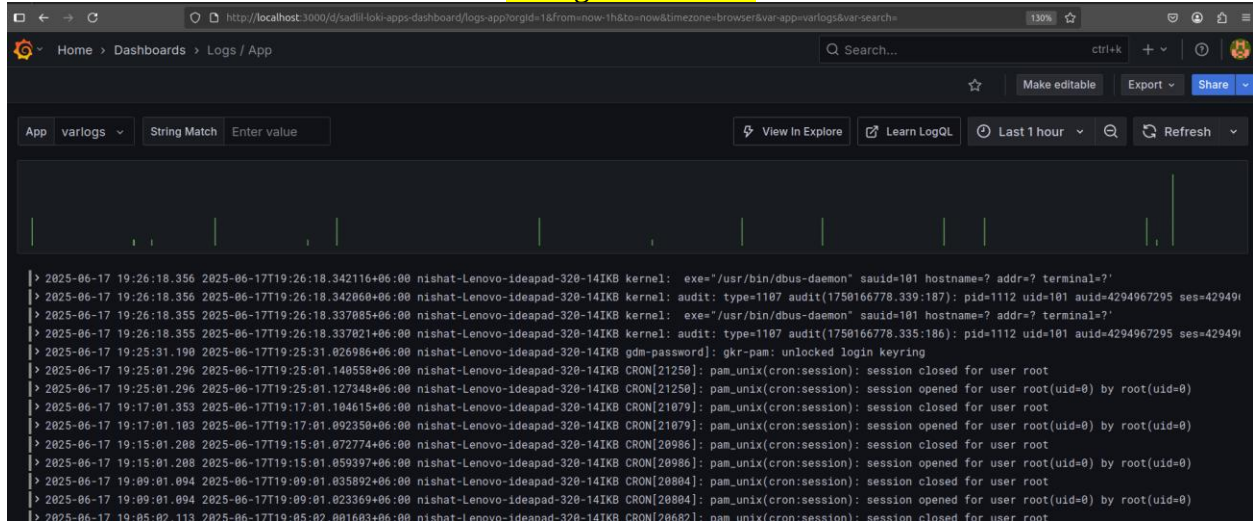
Linux vm performance metric using Prometheus, Loki, and Grafana

Documented by: NISHAT AFLA

****Real-time monitoring****



Log collection

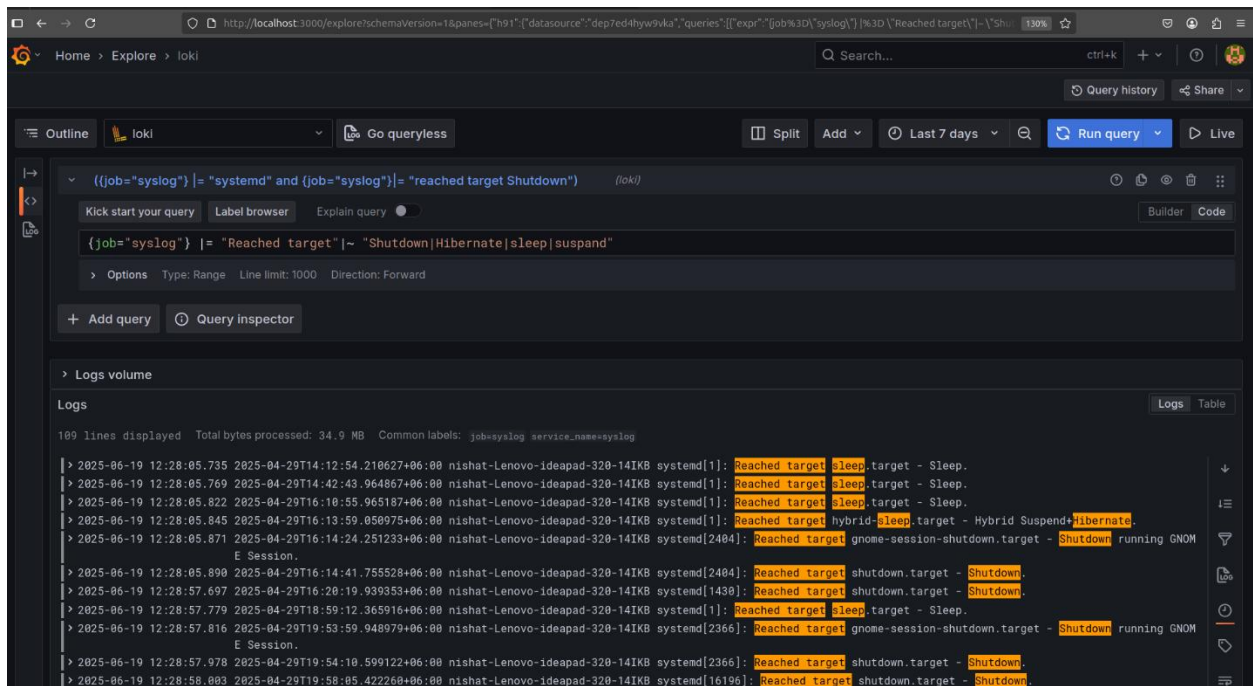


##

Event logs (last 7 days vm on/off events) visualization using grafana:

>**LogQL Query**:

```
{job="syslog"} |= "Reached target" |~ "Shutdown|Hibernate|sleep|suspend"
```



Alerting set up if cpu usage exceed 80%

status: pending

The screenshot shows the Grafana Alerting rules page. The browser address bar indicates the URL is `http://localhost:3000/alerting/list`. The page title is "Alert rules". Below the title, there are filters for "Search by data sources" (set to "All data sources"), "Dashboard" (set to "Select dashboard"), and "State" (set to "Firing", "Normal", "Pending", "Recovering"). The "Rule type" is set to "Alert", "Health" is "Ok", and "Contact point" is "Choose". The "Search" bar is empty. The "View as" dropdown is set to "Grouped".

There are 2 rules: 1 pending, 1 normal. The "Grafana-managed" section shows a list of rules. The first rule is "cpu usage" with a state of "Normal". The second rule is "High CPU Usage Alert" with a state of "Pending" for 32s. The "Data source-managed" section is empty.

State	Name	Health	Summary	Next evaluation	Actions
Normal	cpu usage	ok	CPU usage exceeded 80%	within 30s	View Edit More
Pending for 32s	High CPU Usage Alert	ok	CPU usage on Linux VM exceeded 80%	In a few seconds	View Edit More

status: firing

The screenshot shows the Grafana Alerting rules page. The browser address bar indicates the URL is `http://localhost:3000/alerting/list`. The page title is "Alert rules". Below the title, there are filters for "Search by data sources" (set to "All data sources"), "Dashboard" (set to "Select dashboard"), and "State" (set to "Firing", "Normal", "Pending", "Recovering"). The "Rule type" is set to "Alert", "Health" is "Ok", and "Contact point" is "Choose". The "Search" bar is empty. The "View as" dropdown is set to "Grouped".

There is 1 rule: 1 firing. The "Grafana-managed" section shows a list of rules. The first rule is "High CPU Usage Alert" with a state of "Firing" for 4s. The "Data source-managed" section is empty.

State	Name	Health	Summary	Next evaluation	Actions
Firing for 4s	High CPU Usage Alert	ok	CPU usage on Linux VM exceeded 80%	In a few seconds	View Edit More

SETUP

Install Node Exporter (System Metrics Collector)

Node Exporter collects CPU, RAM, disk, and network metrics from your Linux VM and exposes them to Prometheus.

Download Node Exporter

```
`cd /opt`
```

```
`sudo wget
```

```
https://github.com/prometheus/node_exporter/releases/download/$latest_version/node_exporter-  
${latest_version#v}.linux-amd64.tar.gz  
`
```

****Extract and Move Binary****

```
`sudo tar -xvf node_exporter-*.tar.gz -C /opt`
```

Move the binary to `/usr/local/bin`:

```
`sudo mv /opt/node_exporter-*.linux-amd64/node_exporter /usr/local/bin/`
```

Check the binary works:

```
`/usr/local/bin/node_exporter --version`
```

create a systemd service so node_exporter starts automatically and runs in the background.

****Create the service file:****

```
`sudo nano /etc/systemd/system/node_exporter.service`
```

****Paste this:****

```
>[Unit]
```

```
Description=Node Exporter
```

```
After=network.target
```

```
[Service]
```

```
User=nobody {replace `User=nobody` with your actual Ubuntu username if different.}
```

```
ExecStart=/usr/local/bin/node_exporter
```

```
[Install]
```

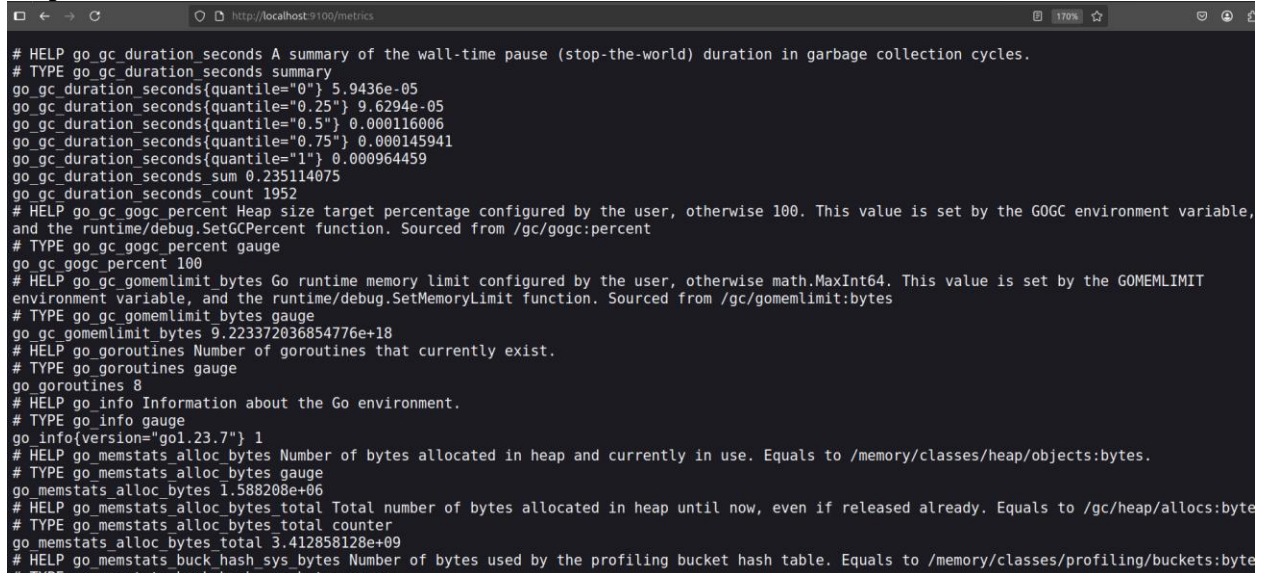
```
WantedBy=default.target
```

****Start and Enable the Service****

```
sudo systemctl daemon-reexec
sudo systemctl daemon-reload
sudo systemctl enable node_exporter
sudo systemctl start node_exporter
```

Verify it's Running

<http://localhost:9100/metrics>



**Install and configure Prometheus to scrape Node Exporter metrics.**

Install Prometheus

```
`cd /opt`
```

****Download the latest valid Prometheus version****

```
`sudo wget https://github.com/prometheus/prometheus/releases/download/v2.52.0/prometheus-2.52.0.linux-amd64.tar.gz`
```

****Extract it****

```
`sudo tar xvf prometheus-2.52.0.linux-amd64.tar.gz`
```

****Go into the directory****

```
`cd prometheus-2.52.0.linux-amd64`
```

```
_`sudo nano prometheus.yml`
```

paste the content:

>global:

scrape_interval: 15s

scrape_configs:

> - job_name: "prometheus"

```
static_configs:
  - targets: ["localhost:9090"]
```

```
> - job_name: "node_exporter"
  static_configs:
    - targets: ["localhost:9100"]
```

****Start Prometheus****

```
>sudo chown -R $ USER:$ USER /opt/prometheus-2.52.0.linux-amd64/data
>./prometheus --config.file=prometheus.yml
```

You should see Prometheus starting logs.

Access Prometheus UI

```
>sudo bash -c "./prometheus --config.file=prometheus.yml > /opt/prometheus.log 2>&1 &"
```

Open your browser and go to:

👉 <http://localhost:9090>

Install Loki (for logs collection)

Loki will collect and store logs — Promtail will send system logs to it.

****Download Loki****

```
cd /opt
latest_loki=$(curl -s https://api.github.com/repos/grafana/loki/releases/latest | grep tag_name |
cut -d '"' -f4)
wget https://github.com/grafana/loki/releases/download/$latest_loki/loki-linux-amd64.zip
```

****Unzip Loki****

```
sudo apt install unzip -y
unzip loki-linux-amd64.zip
chmod +x loki-linux-amd64
```

****Create Loki Config File****

Create the file:

```
nano loki-config.yaml
```

Paste the following content:

```
auth_enabled: false
```

```
server:
```

```
  http_listen_port: 3100
```

log_level: info

ingester:

lifecycle:

ring:

kvstore:

store: inmemory

replication_factor: 1

chunk_idle_period: 5m

max_chunk_age: 1h

wal:

dir: /tmp/wal

schema_config:

configs:

- from: "2022-01-01"

store: boltdb

object_store: filesystem

schema: v11

index:

prefix: index_

period: 24h

storage_config:

boltdb:

directory: /tmp/loki/index

filesystem:

directory: /tmp/loki/chunks

limits_config:

reject_old_samples: true

reject_old_samples_max_age: 168h

allow_structured_metadata: false

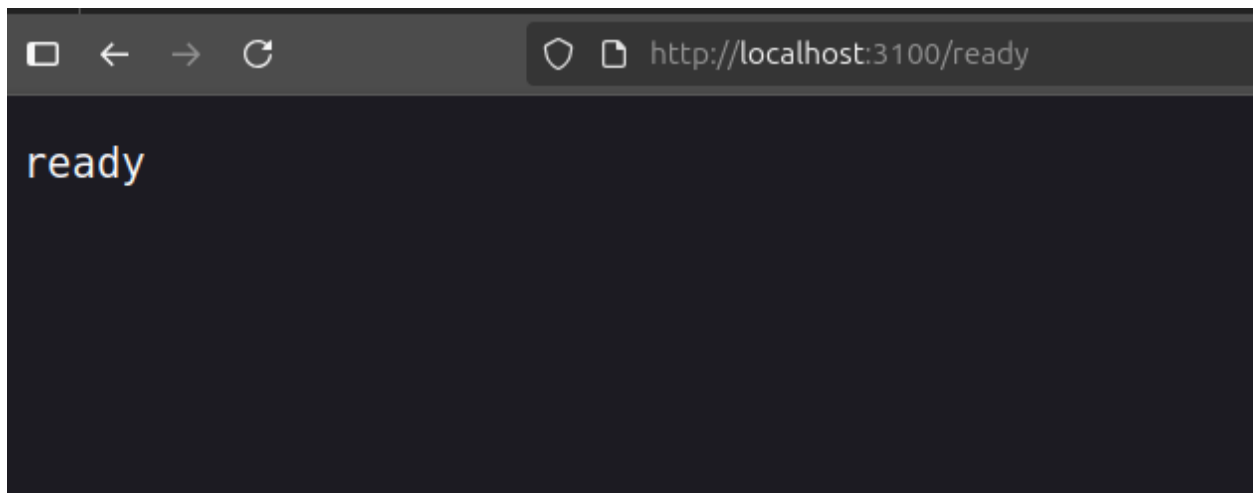
table_manager:

retention_deletes_enabled: true

retention_period: 168h

****Start Loki in background****

>sudo bash -c ". /loki-linux-amd64 -config.file=loki-config.yaml > loki.log 2>&1 &"



****Promtail Setup****

```
cd /opt
```

```
sudo wget https://github.com/grafana/loki/releases/download/v3.5.1/promtail-linux-amd64.zip
```

Then unzip it (also with sudo):

```
sudo unzip promtail-linux-amd64.zip
```

And make it executable:

```
sudo chmod +x promtail-linux-amd64
```

Create a file named `promtail-config.yaml` inside `/opt`:

```
>sudo nano /opt/promtail-config.yaml
```

server:

```
  http_listen_port: 9080
```

```
  grpc_listen_port: 0
```

positions:

```
  filename: /tmp/positions.yaml
```

clients:

```
- url: http://localhost:3100/loki/api/v1/push
```

scrape_configs:

```
- job_name: system-logs
```

```
  static_configs:
```

```
    - targets:
```

```
      - localhost
```

```
    labels:
```

```
      job: varlogs
```

```
      __path__: /var/log/*.log
```


This will collect logs from files like `/var/log/syslog`, `/var/log/auth.log`, etc., and send them to Loki at `localhost:3100`.

run promtail in background:

```
sudo bash -c ". /promtail-linux-amd64 -config.file=promtail-config.yaml > promtail.log 2>&1 &"
```

****Grafana:****

Manually add the GPG key for Grafana:

```
`sudo mkdir -p /etc/apt/keyrings
curl -fsSL https://packages.grafana.com/gpg.key | gpg --dearmor | sudo tee
/etc/apt/keyrings/grafana.gpg > /dev/null`
```

Update your Grafana source to use the keyring:

```
`echo "deb [signed-by=/etc/apt/keyrings/grafana.gpg] https://packages.grafana.com/oss/deb
stable main" | \
sudo tee /etc/apt/sources.list.d/grafana.list`
```

Update package list and install Grafana:

```
`sudo apt-get update`
`sudo apt-get install grafana`
```

Start and enable Grafana service:

```
`sudo systemctl start grafana-server`
`sudo systemctl enable grafana-server`
```

Access Grafana:

Open your browser and go to: `http://localhost:3000`

Default login cred:

Username: admin

Password: admin

****Now it's time to verify**** each component and connect them inside ****Grafana******

Visit this in your browser:

****http://localhost:9090**** → Prometheus web UI should open.

* Click ****“Status” > “Targets”****

* You should see a ****target like localhost:9100**** (Node Exporter)

* ****Health = "UP"**** means Prometheus is pulling metrics from Node Exporter

in Grafana:

1. **Go to (Settings) > Data Sources**

Add datasource as prometheus:

- * **Prometheus**
- * URL: `http://localhost:9090`

Loki

- * URL: `http://localhost:3100`

Click “Save & Test” on both. If successful = ☒

Import Dashboards

Now you can import pre-built dashboards:

1. Go to **Import**

2. Use these Dashboard IDs:

- * **Node Exporter** → ID: `1860`
- * **Loki logs dashboard** → ID: `13639`

3. Select your data source (Prometheus for Node Exporter, Loki for logs)

Done you'll now see live CPU, memory, disk usage + logs!